SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

A) Implement the Breadth First Search algorithm to solve a given problem.
B) Implement the Iterative Depth First Search algorithm to solve the same problem.
C) Compare the performance and efficiency of both algorithms.

CODE

```
from collections import deque

network = {
    'Server': ['Router_A', 'Router_B', 'Router_C', 'Router_D'],

    'Router_A': ['Switch_A'],
    'Switch_A': ['Client_PC'],

    'Router_B': ['Hub_A'],
    'Router_C': ['Hub_B'],
    'Hub_A': ['Client_PC'],
    'Hub_B': ['Client_PC'],

    'Router_D': ['Gateway'],
    'Gateway': ['Firewall'],
    'Firewall': ['Client_PC'],

    'Client_PC': []
}

def bfs_shortest_path(graph, start, destination):
    queue = deque([(start, [start])])
    visited = set([start])
    nodes_explored = 0

    while queue:
        current, path = queue.popleft()
        nodes_explored += 1

        if current == destination:
            return path, nodes_explored

        for neighbor in graph.get(current, []):
            if neighbor not in visited:
```

```
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None, nodes_explored


def iterative_dfs_path(graph, start, destination):
    stack = [(start, [start])]
    visited = set()
    nodes_explored = 0

    while stack:
        current, path = stack.pop()
        nodes_explored += 1

        if current == destination:
            return path, nodes_explored

        if current not in visited:
            visited.add(current)

            for neighbor in graph.get(current, []):
                if neighbor not in visited:
                    stack.append((neighbor, path + [neighbor]))

    return None, nodes_explored


start_node = "Server"
end_node = "Client_PC"

bfs_path, bfs_count = bfs_shortest_path(network, start_node, end_node)
dfs_path, dfs_count = iterative_dfs_path(network, start_node, end_node)

print("----- Breadth First Search -----")
print("Path Found :", " -> ".join(bfs_path))
print("Number of Edges :", len(bfs_path) - 1)
print("Nodes Explored :", bfs_count)

print("\n----- Iterative Depth First Search -----")
```

```
print("Path Found :", " -> ".join(dfs_path))
print("Number of Edges :", len(dfs_path) - 1)
print("Nodes Explored :", dfs_count)
```

OUTPUT

```
----- Breadth First Search -----
Path Found : Server -> Router_A -> Switch_A -> Client_PC
Number of Edges : 3
Nodes Explored : 10

----- Iterative Depth First Search -----
Path Found : Server -> Router_D -> Gateway -> Firewall -> Client_PC
Number of Edges : 4
Nodes Explored : 5
```

SHUBHAM RAORANE